



AI ENGINEERING STACK

Graph RAG Explained

From flat document chunks to connected
knowledge graphs that reason across your entire
data

Swipe to learn it step by step →



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

↻ Repost

What is Graph RAG?

Traditional RAG retrieves isolated text chunks. **Graph RAG builds a knowledge graph** from your documents and retrieves connected entities, relationships, and topic summaries instead of disconnected paragraphs.

💡 CORE IDEA

Nodes = entities. Edges = relationships. Retrieval = graph traversal, not just vector similarity.

✓ WHAT IT ENABLES

- Multi-hop reasoning
- Cross-document answers
- Thematic summaries
- Relationship tracing

🔍 TWO QUERY MODES

- **Local** specific facts
- **Global** broad themes
- Combined answers

Pipeline preview:

Documents → Extract Entities → Build Graph
Communities → Dual Index → Route Query → Answer



RAG retrieves chunks. Graph RAG retrieves knowledge.



AI Coach John

Follow



PROITERIDGE
Strive For Better Future

↺ Repost

Traditional RAG vs Graph RAG

TRADITIONAL RAG

GRAPH RAG

DATA STRUCTURE

— Flat text chunks

✓ Nodes + edges graph

RETRIEVAL METHOD

— Vector similarity only

✓ Graph traversal + vector

MULTI-HOP REASONING

✗ Not supported

✓ Built in

GLOBAL UNDERSTANDING

✗ Weak, misses themes

✓ Community summaries

CROSS-DOCUMENT LINKS

✗ Each chunk isolated

✓ Relationships explicit

INDEXING COST

✓ Low, fast setup

— Higher, one-time LLM cost

↑ Graph RAG answers questions traditional RAG cannot even attempt.



AI Coach John

Follow



PROITERIDGE
Strive For Better Future

 Repost

How Graph RAG Works

Two phases: an offline **index phase** (builds the knowledge graph once) and an online **query phase** (retrieves from it in real time).

✂ INDEX PHASE (OFFLINE, RUN ONCE)

- 1 Load, chunk, and clean documents ONCE
- 2 Extract entities and relationships (LLM) ONCE
- 3 Build knowledge graph, detect communities ONCE
- 4 Summarize communities, build dual index ONCE

⚡ QUERY PHASE (REAL-TIME, EVERY REQUEST)

- 5 Validate and sanitize user query LIVE
- 6 Route: Local (specific) or Global (thematic) LIVE
- 7 Retrieve subgraph or community summaries LIVE
- 8 Generate grounded answer with LLM LIVE

i Index once. Query many times. That is the core efficiency of Graph RAG.



AI Coach John

Follow



PROTIBRIDGE
Strive For Better Future

↻ Repost

Document Ingestion

Graph RAG starts just like traditional RAG. Load your documents, clean the text, and split into chunks. Use **larger chunks (500-800 chars)** so entities appear with enough surrounding context for accurate extraction.

```
from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
import re

loader = PyPDFLoader("knowledge_base.pdf")
docs = loader.load()

for doc in docs: # clean raw text
    doc.page_content = re.sub(r'\s+', ' ', doc.page_content)

splitter = RecursiveCharacterTextSplitter(
    chunk_size=600, chunk_overlap=100)
chunks = splitter.split_documents(docs)
print(f"{len(chunks)} chunks ready for extraction")
```

SUPPORTED LOADERS



PyPDFLoader



WebBaseLoader



CSVLoader



TextLoader



Chunk quality directly affects entity extraction quality.



AI Coach John

Follow



PROITERIDGE
Strive For Better Future

 Repost

Entity and Relationship **Extraction**

For each chunk, use an LLM to extract structured triplets: (subject, predicate, object). These become the nodes and edges of your knowledge graph.

ENTITIES (NODES)

- People, roles
- Organizations
- Technologies
- Concepts, events

↔ RELATIONSHIPS (EDGES)

- uses, enables
- belongs_to
- causes, leads_to
- competes_with

```
extract_prompt = """
Extract all entities and relationships from the text.
ENTITY: name | type
RELATION: source | predicate | target

Text: {text}
"""

# Example output:
# ENTITY: LangChain | technology
# ENTITY: RAG | concept
# RELATION: LangChain | enables | RAG
```



Run this extraction once per chunk across your entire corpus.



AI Coach John

Follow



PROTIBRIDGE
Strive For Better Future

 Repost

Knowledge Graph Construction

Merge all extracted entities and relationships into a single graph. Deduplicate entities that refer to the same concept. Edge weight increases each time two entities co-appear.

```
import networkx as nx

G = nx.Graph()

for entity in all_entities:
    G.add_node(entity["name"], type=entity["type"])

for rel in all_relationships:
    src, tgt = rel["source"], rel["target"]
    if G.has_edge(src, tgt):
        G[src][tgt]["weight"] += 1
    else:
        G.add_edge(src, tgt,
                    relation=rel["predicate"], weight=1)

print(f"{G.number_of_nodes()} nodes, {G.number_of_edges()} edges")
```

🔧 LOCAL (PROTOTYPE)

NetworkX in-memory. Fast to set up, no persistence.

🏢 PRODUCTION

Neo4j or Amazon Neptune for scale and persistence.



Community Detection

Use the **Leiden algorithm** to partition the graph into communities: clusters of tightly connected entities. Each community represents a coherent topic cluster from your documents.

```
from graspologic.partition import hierarchical_leiden

communities = hierarchical_leiden(
    graph=G, max_cluster_size=10, random_seed=42)

# Level 0 = finest granularity clusters
community_map = {
    node: cid
    for node, lvl, cid in communities if lvl == 0
}
print(f"Detected {len(set(community_map.values()))} communities")
```

LEVEL 0 (FINE)

Many small clusters. Best for specific entity queries.

LEVEL N (COARSE)

Fewer large clusters. Best for broad thematic answers.



Store summaries at multiple levels for flexible query routing.



AI Coach John

Follow



PROITERIDGE
Strive For Better Future

 Repost

Community Summarization

For each community, use an LLM to generate a short text summary of its entities and relationships. These summaries form the **global index**, enabling broad thematic queries.

```
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser

prompt = PromptTemplate.from_template(
    "Entities: {entities}\n"
    "Relationships: {relationships}\n"
    "Summarize this topic cluster in 2-3 sentences."
)
chain = prompt | llm | StrOutputParser()

community_summaries = {
    cid: chain.invoke({"entities": ents, "relationships": rels})
    for cid, ents, rels in build_community_inputs(G, community_map)
}
```

⚡ WHY THIS MATTERS

- Answers "What are the main themes?" questions
- Captures topic structure invisible in raw chunks
- Enables hierarchy-aware retrieval



Store summaries as vector embeddings. This is your global index.



AI Coach John

Follow



PROITERIDGE

Strive For Better Future

 [Repost](#)

Dual Indexing

Graph RAG maintains two complementary vector indexes. The **local index** handles specific entity lookups. The **global index** handles broad thematic queries via community summaries.

```
from langchain_community.vectorstores import FAISS
from langchain_huggingface import HuggingFaceEmbeddings

embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-mpnet-base-v2")

# Local index: original text chunks
local_index = FAISS.from_documents(chunks, embeddings)
local_index.save_local("local_index")

# Global index: community summaries
global_index = FAISS.from_texts(
    list(community_summaries.values()), embeddings)
global_index.save_local("global_index")
```

LOCAL INDEX

Text chunks. Answers
"who/what/when" questions.

GLOBAL INDEX

Community summaries. Answers
broad theme questions.



Query Routing

Before retrieving, classify the query. **Local** queries ask about specific entities or facts. **Global** queries ask about themes, patterns, or summaries across the whole corpus.

```
def route_query(query: str) -> str:
    prompt = (
        "Is this LOCAL (specific fact) or "
        "GLOBAL (theme/summary)?\n"
        f"Question: {query}\nAnswer: LOCAL or GLOBAL"
    )
    return llm.invoke(prompt).content.strip()

def retrieve(safe_query: str):
    if route_query(safe_query) == "LOCAL":
        return local_index.similarity_search(safe_query, k=6)
    return global_index.similarity_search(safe_query, k=5)
```

🔗 LOCAL EXAMPLES

"What did Company X announce in Q3?" "Who leads the NLP team?"

🌐 GLOBAL EXAMPLES

"What are the main AI trends?" "Summarize the key themes."



Wrong routing = wrong index = poor answers. Tune your classifier.



AI Coach John

Follow



PROITERIDGE
Strive For Better Future

🔄 Repost

Answer Generation

Combine retrieved graph context with the user query. The LLM synthesizes entities, relationships, and community summaries into a single grounded answer.

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

prompt = ChatPromptTemplate.from_messages([
    ("system", "Answer ONLY from the knowledge graph context. "
               "State clearly if the answer is not found."),
    ("human", "Context:\n{context}\n\nQuestion: {question}")
])

chain = prompt | llm | StrOutputParser()
docs = retrieve(safe_query)

answer = chain.invoke({
    "context": "\n\n".join(d.page_content for d in docs),
    "question": safe_query
})
```

SUPPORTED LLMs



ChatGroq (free)



ChatOpenAI



ChatAnthropic



ChatOllama (local)



AI Coach John

Follow



PROITERIDGE
Strive For Better Future

 Repost

When to Use **Graph RAG**

✓ **USE GRAPH RAG WHEN**

- ✓ Questions span multiple documents
- ✓ Rich entity relationships exist
- ✓ Users ask broad thematic questions
- ✓ Multi-hop reasoning is needed
- ✓ Explainability matters
- ✓ Large corpus, 100+ documents

✗ **STICK WITH RAG WHEN**

- ✗ Simple single-doc lookups
- ✗ Speed and cost are critical
- ✗ Data has no entity structure
- ✗ Small dataset under 100 docs
- ✗ Real-time indexing required

BEST INDUSTRY FITS



Healthcare / Clinical



Legal / Compliance



Financial Research



Scientific / R&D



Graph RAG trades higher upfront cost for far better complex query quality.



Al Coach John

Follow



PRO|ITER|BRIDGE
Strive For Better Future

 **Repost**

Full Pipeline **at a Glance**

Every card in this carousel is one step. Here is the complete Graph RAG pipeline.

INDEX PHASE (offline, run once)

1. Load and chunk documents
2. Extract entities + relationships (LLM)
3. Build knowledge graph (nodes + edges)
4. Detect communities (Leiden algorithm)
5. Summarize each community (LLM)
6. Build dual index: local + global

QUERY PHASE (real-time, every request)

7. Validate and sanitize user query
8. Route query: Local or Global
9. Retrieve subgraph or community summaries
10. Generate grounded answer (LLM)



Save this. Share it. Build with it.

Follow for more AI engineering content



AI Coach John

Follow



PROITERIDGE
Strive For Better Future

Repost